SRE Interview Preparation

AWS CI/CD Pipeline

Complete Interview Q&A; Guide

Prepared for: Dilan Vasantharaj

Role target: Site Reliability Engineer — Melbourne, Australia

AWS Services Covered in This Project

GitHub	Source code, webhooks, CI/CD trigger
Jenkins	CI/CD engine running on EC2
Docker	Multi-stage image builds
Amazon ECR	Private Docker image registry
Amazon ECS Fargate	Serverless container runtime
Amazon EC2	Jenkins build server
AWS VPC	Custom private network
Subnets	Public subnets across 2 AZs
Internet Gateway	VPC front door — inbound + outbound
Route Tables	Traffic routing rules
Security Groups	Stateful firewall per resource
IAM Roles	Permissions without passwords
Elastic IP	Permanent static IP for EC2
ALB	Application Load Balancer — stable DNS
Target Group	Health tracking of ECS tasks
ECS Auto Scaling	Scale min 1 max 5 on CPU 60%
Amazon S3	Remote Terraform state storage
CloudWatch	Container logs and metrics
Terraform	Infrastructure as Code — 9 .tf files

AWS IAM	Identity and access management

Difficulty levels: Senior = 3-5yr engineer Lead = 5yr+ / architect Expert = principal / staff

Section 1 — Networking Fundamentals

Core Concepts

IP address — the home address of a computer. Public IP is visible to the internet. Private IP (10.0.x.x) is only visible inside the VPC. Port — the door number on a building. IP gets you to the building, port gets you to the right room. TCP — reliable data delivery with confirmation. SSH — secure remote terminal access. HTTP port 80 unencrypted, HTTPS port 443 encrypted.

Your VPC Design

VPC CIDR	10.0.0.0/16 — 65,536 private IP addresses
Subnet A	10.0.1.0/24 — AZ ap-southeast-2a — Jenkins EC2
Subnet B	10.0.2.0/24 — AZ ap-southeast-2b — ECS Fargate tasks
IGW	Front door of VPC — enables inbound and outbound internet
Route table	0.0.0.0/0 → IGW — this ONE rule makes a subnet public

Q1 [Senior]

What makes a subnet public in AWS?

Model answer:

One thing only — the route table attached to it has a route: 0.0.0.0/0 → Internet Gateway. That single route is the definition. map_public_ip_on_launch just means resources automatically get a public IP but without the IGW route that IP is unreachable. Your EC2 and ECS are in public subnets specifically so they can receive inbound traffic from the internet.

Q2 [Senior]

What is the difference between a public IP and a private IP?

Model answer:

Public IP is visible and reachable from the entire internet. Private IP (10.0.x.x) is only reachable from inside the VPC. In your project the EC2 has both — a private IP for talking to other AWS services inside the VPC and a public IP so GitHub webhooks and your browser can reach Jenkins. ECS tasks also get both when in a public subnet.

Q3 [Senior]

Walk me through how a user request reaches your React app.

Model answer:

Browser types ALB URL. TCP packet travels over internet. Hits Internet Gateway — front door of VPC. IGW routes to the ALB subnet. ALB security group checks: port 80 allowed from 0.0.0.0/0 — yes. ALB selects a healthy ECS task from the target group. ECS security group checks: port 5173 from ALB SG — yes. Packet reaches the container. serve web server responds. Response travels back automatically — security group is stateful so return packets are allowed.

Q4 [Lead]

What is the difference between a security group and a NACL?

Model answer:

Security Group is stateful, works at resource level (per EC2/ECS/RDS), only allow rules no deny. NACL is stateless, works at subnet level, has both allow AND deny rules. Because NACLs are stateless you must explicitly allow both inbound AND outbound for the same connection. Security groups are used for most things. NACLs are an extra layer for subnet-wide rules. Stateful means if your EC2 initiates an outbound call, the return traffic is automatically allowed in.

Q5 [Lead]

What is an Elastic IP and why did you add it to Jenkins?

Model answer:

Elastic IP is a static public IP address that persists regardless of EC2 state. Normal EC2 public IPs change every time the instance restarts or is replaced. This is a problem because the GitHub webhook URL needs to stay constant. With Elastic IP the Jenkins URL http://13.239.149.119:8080 never changes even if the EC2 is destroyed and recreated with Terraform. I also added lifecycle prevent_destroy to protect the EIP from accidental deletion.

Section 2 — EC2 & IAM Roles

Instance type	t3.micro — 1 vCPU 1GB RAM — free tier in ap-southeast-2
AMI	Amazon Linux 2023 — minimal, AWS CLI pre-installed
Java version	Java 21 Amazon Corretto — Jenkins dropped Java 17 support
IAM role	jenkins-ec2-role — ECR push + ECS deploy + SSM access
Key pair	jenkins-key2.pem — RSA key for SSH access
/tmp fix	mount -o remount,size=4G — minimal AMI has only 458MB /tmp
Swap	2GB swapfile — Jenkins on 1GB RAM needs swap to avoid OOM

Q6 [Senior]

What is the difference between an IAM Role and an IAM User?

Model answer:

IAM User is a person. Has a username, password, and permanent access keys. You log in as a user. IAM Role is a set of permissions that a SERVICE picks up temporarily. No password. No permanent keys. The EC2 assumes the role and gets short-lived credentials that rotate every few hours automatically. This is why roles are safer — there are no long-lived secret keys that can be leaked in code. In your Jenkins pipeline, never use aws configure with access keys. Use the instance role instead.

Q7 [Senior]

How does EC2 push to ECR without storing any credentials?

Model answer:

IAM Instance Role. The EC2 has jenkins-ec2-role attached with AmazonEC2ContainerRegistryFullAccess. When Jenkins runs aws ecr get-login-password, the AWS CLI calls the EC2 Instance Metadata Service at 169.254.169.254 to get temporary credentials from that role. Credentials rotate automatically every few hours. Nothing is stored in the Jenkinsfile, environment variables, or any config file. If the EC2 is compromised the attacker only has the limited permissions of that role.

Q8 [Lead]

What is the difference between an IAM role and an instance profile?

Model answer:

You cannot attach an IAM role directly to an EC2 instance. EC2 needs a wrapper called an Instance Profile. The instance profile holds the role and the instance profile gets attached to the EC2. In the AWS Console this is hidden — when you click attach IAM role it creates the instance profile silently. In Terraform you must create aws_iam_instance_profile separately. The role is the permission card, the instance profile is the lanyard, the EC2 wears the lanyard.

Q9 [Senior]

What is least privilege and why does it matter?

Model answer:

Least privilege means giving a resource only the exact permissions it needs and nothing more. In your project Jenkins uses AmazonEC2ContainerRegistryFullAccess — fine for learning but in production you would scope it to only allow ecr:PutImage on the specific repository ARN. Why it matters: if the EC2 is compromised the attacker can only do what the role allows. With FullAccess they could delete all your ECR images. With a scoped policy they can only push to one repo.

Q10 [Expert]

A developer wants to store AWS access keys in the Jenkinsfile. What do you say?

Model answer:

No. Access keys in a Jenkinsfile are stored in Git history forever — even if you delete them later they exist in previous commits. Anyone with repo access has your AWS credentials. Bots scan GitHub 24/7 and can find and use leaked keys within minutes to spin up EC2 instances for crypto mining. The correct approach is an IAM role attached to the EC2 instance running Jenkins. Jenkins inherits temporary credentials from that role automatically. No keys needed anywhere. Credentials rotate every few hours. This is non-negotiable.

Section 3 — Docker & Amazon ECR

Multi-stage Dockerfile

Stage 1 (builder): FROM node:18-alpine, copies source, runs npm install and npm run build, produces dist/ folder. Stage 2 (runtime): starts fresh from node:18-alpine, copies ONLY dist/, installs serve web server, CMD runs serve -s dist -l 5173. Final image has no source code, no node_modules — small and secure.

ECR repo	cicd-react-app — private Docker registry in ap-southeast-2
Scan on push	Enabled — every image scanned for CVE vulnerabilities automatically
Lifecycle policy	Keep last 10 images, expire older — prevents storage cost growth
Image tag	MUTABLE — :latest tag overwritten on each Jenkins build

Q11 [Senior]

What is a multi-stage Docker build and why did you use it?

Model answer:

Multi-stage build uses multiple FROM statements in one Dockerfile. Stage 1 (builder) installs all Node.js tooling and builds the React app with Vite producing the dist/ folder. Stage 2 (runtime) starts completely fresh and copies only the dist/ folder plus a lightweight serve package. The final image has no source code, no node_modules, no build tools — just the compiled static files. This makes the image smaller and more secure. No source code is ever exposed in the running container.

Q12 [Senior]

Why do you copy package.json before copying source code in the Dockerfile?

Model answer:

Docker layer caching. Each RUN and COPY instruction creates a layer. Docker caches layers and only rebuilds from the first changed layer downward. If you copy package.json first then run npm install, that layer is cached as long as package.json does not change. On the next build when only App.jsx changed, Docker reuses the npm install cache and only reruns COPY and npm run build. If you copied all files first, any code change would invalidate the npm install cache and reinstall all packages every build — wasting 60 seconds.

Q13 [Senior]

What is ECR and how does ECS authenticate with it to pull images?

Model answer:

ECR is Amazon Elastic Container Registry — a private Docker image registry managed by AWS. The ECS Task Execution Role handles authentication. This is a separate IAM role (ecs-task-execution-role) attached in the task definition. It needs ecr:GetAuthorizationToken and ecr:BatchGetImage permissions. When ECS starts a new task it automatically uses this role to get a token from ECR and pull the image. You write no authentication code anywhere.

Q14 [Lead]

Why does ECR not have a security group?

Model answer:

Security groups only attach to resources that live INSIDE your VPC — EC2 instances, ECS tasks, RDS databases. ECR is an AWS managed service that lives OUTSIDE your VPC. AWS controls the infrastructure. Access to ECR is controlled by IAM policies not network rules. Your EC2 and ECS tasks reach ECR by making outbound HTTPS calls on port 443 through the Internet Gateway. The IAM role is what determines whether the call is authorized — not a security group.

Section 4 — ECS Fargate

Cluster	cicd-cluster — logical grouping, no servers with Fargate
Task CPU	256 units (0.25 vCPU)
Task memory	512 MB — increase to 1024 if exit code 137 OOM errors occur
Network mode	awsvpc — each task gets its own ENI and private IP in the VPC
Port	5173 — serve static React files
Log driver	awslogs → /ecs/cicd-react-app CloudWatch 7-day retention
Exec role	ecs-task-execution-role — ECR pull + CloudWatch write

Q15 [Senior]

What is the difference between ECS EC2 launch type and Fargate?

Model answer:

EC2 launch type: you manage the underlying EC2 instances — patching OS, choosing instance types, scaling the server fleet, paying for EC2 whether tasks run or not. Fargate: AWS manages everything below the container — no servers to manage, no OS patching, you only define CPU and memory needed, you pay per second the container actually runs. For this project Fargate is correct because there is one app and managing EC2 instances just to run containers adds operational overhead without value.

Q16 [Senior]

After Jenkins pushes a new image to ECR how does ECS pick it up?

Model answer:

ECS does not watch ECR automatically. You must trigger a redeployment. Jenkins runs: aws ecs update-service --cluster cicd-cluster --service cicd-react-app-service --force-new-deployment. This tells ECS to stop the old task and start a new one. The new task pulls :latest from ECR which is the image Jenkins just pushed. Without --force-new-deployment ECS keeps running the old container forever even after you push a new image.

Q17 [Lead]

Your ECS task keeps stopping with exit code 137. What does that mean and how do you fix it?

Model answer:

Exit code 137 is SIGKILL — the container was killed by the Linux OOM killer. The container tried to use more memory than the task definition allowed. Investigation: check CloudWatch Logs at /ecs/cicd-react-app for errors before shutdown, check ECS service events for 'OutOfMemory: Container killed due to memory usage', check CloudWatch Container Insights metrics for memory hitting 100%. Fix: increase memory in the task definition — try doubling from 512MB to 1024MB. Also check the application for memory leaks.

Q18 [Lead]

Why does the ECS task need an outbound security group rule?

Model answer:

On startup, ECS Fargate must pull the Docker image from ECR — this is an outbound HTTPS call to ECR on port 443. If the egress rule is missing the call is blocked at the network level and the task fails immediately with CannotPullContainerError. The task also needs outbound access to send logs to CloudWatch. This is one of the most common ECS debugging questions — junior engineers try to add inbound rules for ECR which is wrong. ECR is not calling you, you are calling ECR. That is outbound.

Q19 [Expert]

What happens at 3am when your ECS task crashes?

Model answer:

ECS service detects running count dropped below desired count (desired=1, running=0). It immediately starts a replacement task. The replacement task pulls :latest from ECR — takes 30-60 seconds. During that time the app is down. To prevent downtime: set desired_count=2 and configure minimum_healthy_percent=50 maximum_percent=200. ECS starts the new task before stopping the old one. With ALB the health check detects the crash within 30 seconds and stops routing traffic to the bad task. The other task continues serving all traffic.

Section 5 — ALB & Auto Scaling

ALB type	Application Load Balancer — layer 7 HTTP/HTTPS
Listener	Port 80 HTTP → forwards to target group
Target group	cicd-app-tg — tracks ECS task IPs, health check every 30s
Target type	IP — required for Fargate (each task has own ENI)
Health check	GET / port 5173, 200 OK = healthy, 2 pass = in rotation
Scale trigger	CPU average 60% across all tasks
Min/max tasks	Minimum 1, maximum 5
Scale out	60 second cooldown — fast response to traffic spikes
Scale in	300 second cooldown — conservative removal

Q20 [Senior]

Why did you add an ALB instead of exposing ECS directly?

Model answer:

Three reasons. First, stable URL — ECS task public IPs change on every deployment. ALB DNS name never changes. Second, health checking — ALB checks every task every 30 seconds and only routes to healthy ones. If a task crashes ALB stops sending traffic there within 30 seconds. Third, security — with ALB the ECS security group only allows traffic from the ALB security group not from the internet directly. Users cannot bypass the ALB. Future HTTPS, WAF, and rate limiting all terminate at the ALB.

Q21 [Senior]

Why is target_type = ip required for Fargate?

Model answer:

Fargate uses awsvpc network mode — each task gets its own Elastic Network Interface with its own private IP address. There is no EC2 instance to target. The target group must register the private IP of each running task directly. If you set target_type = instance with Fargate there is nothing to register and the ALB serves 503 to all requests. ECS automatically registers and deregisters task IPs in the target group as tasks start and stop.

Q22 [Lead]

Why is scale_in_cooldown 5x longer than scale_out_cooldown?

Model answer:

Scale out is fast (60 seconds) because traffic spikes need immediate response — users are suffering right now. Scale in is slow (300 seconds) because traffic drops can be temporary. If we remove tasks immediately after a 2-minute dip and traffic spikes again we are back to one task while new tasks take 60 seconds to become available. Waiting 5 minutes ensures the traffic drop is genuine before removing capacity. The rule: always be generous adding capacity, always be conservative removing it. This prevents yo-yo scaling.

Q23 [Lead]

What happens when an ECS task fails a health check?

Model answer:

ALB sends HTTP GET to port 5173 every 30 seconds. If the task returns anything other than 200 OK or times out after 5 seconds it counts as one failure. After 3 consecutive failures (unhealthy_threshold=3) the task is marked unhealthy. ALB immediately stops routing new requests to that task. Existing connections complete normally. ECS service detects the unhealthy task and starts a replacement. Once the new task passes 2 consecutive health checks (healthy_threshold=2) it enters the target group and starts receiving traffic. With 2+ tasks running user downtime is zero.

Section 6 — Terraform & Infrastructure as Code

vpc.tf	VPC, 2 subnets, Internet Gateway, route table, associations
security.tf	Jenkins SG, ALB SG, ECS SG (ALB only), egress rules
ec2.tf	IAM role, instance profile, EC2, Elastic IP, user data
ecr.tf	ECR repository, lifecycle policy keep last 10 images
ecs.tf	Cluster, task execution role, CloudWatch log group, service with ALB
alb.tf	ALB, ALB SG, target group with health checks, listener port 80
autoscaling.tf	App Auto Scaling target and CPU target tracking policy
variables.tf	region, instance_type, your_ip, key_pair_name
outputs.tf	jenkins_public_ip, ecr_repository_url, alb_dns_name

Q24 [Senior]

What does terraform plan do?

Model answer:

terraform plan compares your .tf files (desired state) against the state file (current real-world state) and shows you a diff — what will be created (+), changed (~), or destroyed (-). Nothing is changed in AWS. It is your chance to review before committing. Always run plan before apply. In teams you run plan in CI/CD and only apply on merge to main branch.

Q25 [Senior]

What is the Terraform state file and why does it matter?

Model answer:

The state file (terraform.tfstate) maps your .tf resource blocks to real AWS resource IDs. Without it Terraform cannot know what it has already created and would try to create everything again. Stored in S3 for teams — enables multiple engineers to use Terraform without conflicts, protects state if your laptop dies, and encrypts sensitive values. Never manually delete AWS resources managed by Terraform — this creates state drift and next terraform plan will try to recreate the deleted resource.

Q26 [Lead]

What happens if two engineers run terraform apply at the same time?

Model answer:

Without state locking both operations read the current state simultaneously, compute changes, and try to write back — last write wins resulting in corrupted state and potentially conflicting AWS resources. Fix: S3 backend with DynamoDB locking. Before any operation Terraform writes a LockID record to DynamoDB. If the record exists the second operation fails with 'Error locking state: state file locked'. When apply finishes the lock is released. In CI/CD pipelines also serialise Terraform runs using pipeline gates.

Q27 [Lead]

Why do you need lifecycle ignore_changes on task_definition in the ECS service?

Model answer:

Without ignore_changes every Jenkins deployment updates the ECS task definition to a new revision. When you next run terraform apply Terraform sees the current task definition does not match what it originally created and tries to revert it — fighting with your CI/CD pipeline. ignore_changes = [task_definition] tells Terraform: deployments are Jenkins's job not yours. Terraform manages the service configuration but lets Jenkins manage the running version.

Q28 [Expert]

What is the chicken-and-egg problem with Terraform backends?

Model answer:

Terraform needs the S3 backend bucket to store state. But Terraform is also what creates things. You cannot use Terraform to create its own backend — Terraform needs the backend to exist before it can run. Solution: create the S3 bucket and DynamoDB table manually using AWS CLI before running terraform init. Then add backend.tf pointing to that bucket. Then run terraform init. This is why the S3 bucket for state must be created first before any other Terraform commands.

Section 7 — CI/CD Pipeline & Jenkins

4 Pipeline Stages

Stage 1 — Install	npm install inside react-app folder — downloads dependencies
Stage 2 — Docker build	docker build reads Dockerfile, creates image on EC2
Stage 3 — ECR push	login via IAM role, tag with ECR URI, docker push
Stage 4 — ECS deploy	aws ecs update-service --force-new-deployment

Q29 [Senior]

What is the difference between CI and CD?

Model answer:

CI (Continuous Integration) — every code commit automatically triggers build and tests. Goal is to catch bugs early by integrating frequently. CD (Continuous Deployment) — every successful CI run automatically deploys to production. Goal is speed with zero manual steps. Your pipeline does both: push → built → deployed automatically. Most teams add a manual approval gate before production — this middle ground is called Continuous Delivery.

Q30 [Senior]

How does the GitHub webhook trigger Jenkins?

Model answer:

GitHub is configured with a webhook URL pointing to http://jenkins-ip:8080/github-webhook/. Every git push fires a POST request to that URL. Jenkins has the GitHub plugin installed and the pipeline job has GitHub hook trigger for GITScm polling enabled. Jenkins receives the POST, matches it to the configured repository, and starts a new build. The trailing slash in the URL is required — without it Jenkins returns a 302 redirect and the webhook fails.

Q31 [Lead]

Jenkins fails with git command not found. What is wrong?

Model answer:

Git is not installed on the EC2. Jenkins uses the system git to clone your repository. Fix: dnf install -y git on the EC2, then configure the git path in Jenkins: Manage Jenkins → Tools → Git installations → set path to /usr/bin/git. Also add git to userdata.sh so it is installed automatically on future EC2 rebuilds: dnf install -y wget git nodejs npm. This is the operational lesson — userdata.sh must include every package Jenkins needs.

Q32 [Expert]

How would you implement zero-downtime deployments in your ECS setup?

Model answer:

Set desired_count=2 and configure deployment settings: minimum_healthy_percent=50 and maximum_percent=200. This means ECS starts one new task (2 running temporarily) before stopping one old task — traffic always has somewhere to go. Add an ALB with target group health checks so traffic only routes to tasks that pass health checks. The ALB waits for the new task to pass 2 consecutive health checks before sending it any traffic. The old task only stops after the new task is confirmed healthy.

Section 8 — Observability & Incident Response

The 3 pillars of observability: Logs (what happened), Metrics (how is it performing), Traces (where did the request go). Your project uses CloudWatch Logs for container output and CloudWatch Metrics for ECS CPU and memory.

Log group	/ecs/cicd-react-app — all container stdout/stderr
Retention	7 days — configurable, costs increase with longer retention
Log driver	awslogs — configured in task definition logConfiguration block
Key commands	journalctl -u jenkins — Jenkins service logs on EC2
Key commands	cat /var/log/cloud-init-output.log — userdata.sh execution log

Q33 [Senior]

Your ECS app went down at 3am. Walk through your investigation.

Model answer:

First acknowledge the alert. Check ECS service events — the stopped reason field shows why the task stopped. Common: CannotPullContainerError (ECR auth or image not found), OutOfMemory (exit code 137), application crash (exit code 1). Check CloudWatch Logs at /ecs/cicd-react-app for errors in the minutes before the crash. Check CloudWatch Metrics for CPU and memory spikes. Once resolved force a new deployment: aws ecs update-service --force-new-deployment. Write a post-incident review within 24 hours.

Q34 [Senior]

Jenkins fails with CannotPullContainerError. What do you check?

Model answer:

In order: (1) Does the image exist in ECR? Run aws ecr list-images --repository-name cicd-react-app. If empty the pipeline did not push successfully. (2) Is the task execution role attached and does it have ECR pull permissions? Check the task definition execution_role_arn. (3) Is there an outbound security group rule on the ECS SG? ECS needs port 443 outbound to reach ECR. (4) Is DNS hostname resolution enabled in the VPC? Required for ECS to resolve the ECR endpoint hostname.

Q35 [Lead]

Design a monitoring strategy for this project.

Model answer:

Three levels. Pipeline health: Jenkins build success rate, build duration trend, ECR push failures. Alert on any failed build or build exceeding 10 minutes. Container health: ECS running task count (alert if below desired), CPU and memory utilisation (alert at 80%), task restart count. Alert on running count below 1 meaning app is completely down. Application health: HTTP 5xx error rate, response latency p99, request rate — needs ALB access logs. CloudWatch dashboard combining all three levels for single pane of glass. All alerts to Slack and PagerDuty for after-hours.

Section 9 — Real Incidents From This Build

These are actual problems you encountered and solved. Use them as STAR format stories in interviews.

Q36 [Senior]

You hit a package conflict on Amazon Linux 2023 installing curl. How did you resolve it?

Model answer:

Amazon Linux 2023 ships with curl-minimal pre-installed. Running dnf install -y curl failed with dependency conflict because full curl and curl-minimal cannot coexist. The error message listed every conflicting version and suggested --allowerasing. Rather than replacing a system package I identified that Jenkins only needs wget not curl — for downloading the Jenkins repo file. I changed the install command to dnf install -y wget only. Root cause fixed in userdata.sh so it never happens on future EC2 builds. Lesson: always read the error message carefully.

Q37 [Senior]

Jenkins was installed but failing to start with exit-code 1. How did you find the root cause?

Model answer:

Ran journalctl -u jenkins -n 30 which showed the exact error: 'Running with Java 17 which is older than the minimum required version Java 21. Supported versions: [21, 25]'. Jenkins released a new version dropping Java 17 support. Fix: dnf install -y java-21-amazon-corretto then alternatives --set java to point to Java 21. Updated userdata.sh to install java-21-amazon-corretto. Key lesson: software version requirements change. Monitor deprecation notices and test infrastructure changes in dev before production.

Q38 [Lead]

Your Elastic IP kept changing even though EIP is supposed to be permanent. Why?

Model answer:

When running terraform destroy -target=aws_instance.jenkins Terraform also destroyed aws_eip_association.jenkins (the attachment). On the next apply something caused the old EIP to be released and a new one created. Two fixes: (1) Add lifecycle prevent_destroy = true to aws_eip so Terraform refuses to destroy it ever. (2) Output from aws_eip.jenkins.public_ip not aws_instance.jenkins.public_ip — the instance has its own public IP separate from the EIP. Always output the EIP address.

Q39 [Lead]

You got 'Remote host identification has changed' when SSH-ing to the EC2. What happened?

Model answer:

The EC2 was destroyed and recreated — same Elastic IP but different server. SSH remembers a fingerprint for each known host in ~/.ssh/known_hosts. The new EC2 has a different host key so SSH flags a potential man-in-the-middle attack. This is a security feature working correctly. Since we know why it changed: run ssh-keygen -R your-ip to remove the old fingerprint, then SSH again and type yes to accept the new fingerprint. In production with Auto Scaling you would use AWS Systems Manager Session Manager instead of SSH to avoid this entirely.

Section 10 — Quick Reference

One-minute project answer — memorise this

> *"I built a fully automated CI/CD pipeline on AWS from scratch. All infrastructure is Terraform — VPC, subnets, security groups, EC2 for Jenkins, ECR for Docker images, ECS Fargate for containers, ALB for load balancing, and Auto Scaling. When I push code to GitHub a webhook triggers Jenkins. Jenkins builds the React app, creates a Docker image using a multi-stage Dockerfile, pushes to ECR using an IAM role — no credentials stored anywhere — then runs aws ecs update-service to deploy. Users access the app through the stable ALB DNS name on port 80. If CPU exceeds 60% auto scaling adds tasks up to maximum 5. The entire infrastructure can be recreated with two commands."*

Career gap answer — say this with confidence

> *"During my relocation to Australia I used the time to build production-grade AWS projects from scratch — including a fully automated CI/CD pipeline with Jenkins, Docker, ECR, ECS Fargate, ALB, Auto Scaling, and Terraform. I am more technically sharp now than before the gap and can apply this knowledge immediately."*

Ports to memorise

22	SSH — remote server login
80	HTTP — unencrypted web traffic
443	HTTPS — encrypted web traffic (ECR, ECS, CloudWatch, GitHub all use this)
8080	Jenkins UI and GitHub webhooks
5173	Your React app (Vite/serve)

Key commands

terraform plan	Dry run — shows changes without touching AWS
terraform apply	Create or update infrastructure
terraform destroy -target=X.Y	Destroy one resource only
terraform output	Print all output values
aws sts get-caller-identity	Verify which account and role CLI is using

aws ecr get-login-password | docker login	Authenticate Docker with ECR via IAM role
aws ecs update-service --force-new-deployment	Force ECS to pull new image
aws elbv2 describe-target-health	Check which ECS tasks are healthy
journalctl -u jenkins -n 30	Check Jenkins service errors
sudo cat /var/log/cloud-init-output.log	Check userdata.sh execution log
ssh-keygen -R IP	Remove old SSH host fingerprint
free -h	Check RAM and swap usage
df -h	Check disk space usage

Dilan Vasantharaj — SRE — Melbourne, Australia — github.com/Dilan8 — vasandarajdilan64@gmail.com